Writing to Files

Implementing FileWriter and Error Handling

codehive

Using FileWriter

The `FileWriter` class is used to write character data to a file. It creates a new file if it doesn't exist, and by default, it **overwrites** existing file content.

```java
import java.io.FileWriter;
import java.io.IOException;

public class WriteToFile {
  public static void main(String[] args) {
    try {
      FileWriter myWriter = new FileWriter("filename.txt");
      myWriter.write("Hello, this is a file!");
      myWriter.close(); // Crucial step
      System.out.println("Successfully wrote to the file.");
    } catch (IOException e) {
      System.out.println("An error occurred.");
      e.printStackTrace();
    }
  }
}
```

Key Components

1. The Import Statement

You must import `java.io.FileWriter` and `java.io.IOException`. Almost all file operations in Java throw checked exceptions, so error handling is mandatory.

2. The .close() Method

When you use `FileWriter`, the data is often stored in a temporary buffer. Calling `.close()` ensures that the data is "flushed" out of memory and into the physical file. It also releases the file lock so other programs can access it.

> **Pro Tip:** While manual `.close()` is shown here, it is better practice to use **Try-with-Resources** (covered in our previous guide) to automate this and prevent memory leaks.

3. Appending Content

To avoid overwriting a file and instead add content to the end of it, use the second boolean parameter in the constructor:

```
// The 'true' parameter enables append mode
FileWriter myWriter = new FileWriter("filename.txt", true);
myWriter.write("Adding a new line here!\n");
```


Data Persistence with

codehive

Mastering Storage streams.